

Simple user manual

Keep each Mac and GitHub in sync. Use sync for everyday work.

1. Before you start

Run as your normal user, not with sudo. You need Git, Python 3, rsync and working GitHub SSH access. Quit Moom before syncing, and avoid editing managed files while a sync is running or awaiting recovery.

```
macos-config-sync.sh version
macos-config-sync.sh status
```

2. Choose the right starting point

Upgrading from v3.1.0: finish or cancel pending work with the old script first. Your existing baseline is retained, so no new adopt is needed. Review managed directories before the first sync: new eligible files now upload automatically.

Already-synchronised Mac upgrading from v3.0.x: use adopt once, only if the clean checkout represents the configuration last deployed to this Mac. It records that baseline without copying files or contacting GitHub.

```
macos-config-sync.sh adopt
macos-config-sync.sh sync
```

New Mac: init prepares the checkout without applying it. Back up valuable local settings separately, then use pull only if you intend the repository to replace the Mac's managed configuration. Do not adopt a fresh clone.

```
macos-config-sync.sh init
macos-config-sync.sh pull
```

Warning: pull and restore can overwrite files and delete local-only files inside managed directories. They are recovery commands, not normal ways to sync.

3. Everyday use

```
macos-config-sync.sh sync
```

Run this after changing your configuration and when switching Macs. It captures local changes, reconciles them with GitHub, asks before propagating deletions, then pushes and applies the result. It backs up local files before deployment. The push command is an alias for sync.

If nothing changed, it skips backup, deployment and Moom import. An outstanding NAS mirror is still retried. Timestamp-only differences are not configuration changes.

4. Configure once, then just sync

Defaults include all eligible files under ~/docs, ~/scripts and ~/.config/git, recursively. Put the PDFs in ~/docs and run sync. No add command is needed. Selected shell, Git, GPG and SSH settings are also managed.

Install the supplied config template as ~/.config/macos-config-sync/config with permissions 600. Edit its MANAGED_DIRECTORIES array to change shared scopes. Omitted arrays keep defaults. The file stays local to each Mac and is parsed as data, not shell code.

Brewfile, installed-apps.txt and Moom.plist are machine-specific. Keep the machine name stable. Sync does not install packages. Known credentials and caches are excluded, but scanning cannot detect every confidential document. Only manage folders you intend to share.

Files and recovery

Make deliberate changes. If a safety check stops the script, inspect before retrying.

5. Optional manual enrolment and exclusions

For tighter control, set `EXPLICIT_DIRECTORIES=("scripts")` in your config. New files in that directory then need add, while docs remains automatic. `AUTO_ADD=0` makes all directories manual. Existing tracked files stay managed. Paths are HOME-relative, regardless of your current directory.

```
macos-config-sync.sh add scripts/example.sh
macos-config-sync.sh sync
```

To keep the local file but remove its repository copy on the next sync:

```
macos-config-sync.sh forget scripts/example.sh
macos-config-sync.sh sync
```

`forget` stays effective in automatic mode until you add the file again. Its repository deletion can affect other Macs. For more exclusions, set `EXTRA_EXCLUDE_PATTERNS=("private/" "*.draft")` in config. Built-in exclusions remain enforced. To delete a managed file everywhere, remove it locally and review the sync deletion prompt. Do not change config during a pending sync.

6. If Git reports a conflict

Resolve conflicts inside the private checkout, not the live HOME files. These commands use the default checkout and an example conflict in `.zshrc`. Substitute the actual conflicted path.

```
repo="$HOME/.local/share/macos-config"
git -C "$repo" status
# Edit the conflicted file: $repo/home/.zshrc
git -C "$repo" add home/.zshrc
git -C "$repo" rebase --continue
macos-config-sync.sh sync
```

Repeat resolution and rebase `--continue` if more conflicts appear. If you want to abandon the attempt, first save any manual resolution edits you need. Run `rebase --abort` only if a rebase is active, then cancel:

```
git -C "$repo" rebase --abort
macos-config-sync.sh cancel
```

`cancel` preserves the pending proposal and resets the private checkout. It never changes HOME, but does not undo commits already pushed or settings already deployed. If HOME changed while a proposal was pending, cancel and start a new sync. Never delete state markers or use `adopt` to silence a safety warning.

7. Backups and occasional commands

Backups: `~/local/state/macos-config/backups/`

The default keeps 10 snapshots. Copy needed files from a chosen snapshot after inspecting it. `restore` does not select a backup snapshot.

restore: apply the clean local checkout without fetching. **nas-push:** mirror repository files to the NAS, including deletions. **nas-pull:** advanced recovery into a missing checkout only, inspect it before use. **DRY_RUN=1:** read-only explanation, not a calculated change preview.

If the secret scanner stops a push, review both files and outgoing commit history. Removing a secret only from the latest file does not remove it from earlier commits. Do not bypass the scan.